

DIA 5

Hugging Face e a solução final

Escolher modelos com critério, avaliar o que foi construído e colocar no ar

1

Hub: escolher com método

2

Tokenização e chat template

3

Pipelines prontos, embeddings
e rerank

4

Avaliação, deploy e o NEXUS
final

As oito horas de hoje

1

09:00 – 10:30 • O Hub e o método de escolha

Como escolher um modelo por requisito, e não por ranking. Tokenização e chat template.

2

10:45 – 12:15 • Pipelines prontos, e laboratório

Sentiment, zero-shot, question answering, summarization. Nem toda decisão precisa de LLM.

3

13:15 – 14:45 • Embeddings, rerank e avaliação

Escolher embedding com critério, rerank em dois estágios, e medir se o sistema presta.

4

15:00 – 16:15 • smolagents, deploy e o NEXUS final

CodeAct, Spaces, um modelo por papel, e o método de depuração em cinco passos.

5

16:15 – 17:00 • Fechamento e projeto final

Os sete princípios, o que o curso não cobriu, e o enunciado do projeto.

01

O Hub e o método de escolha

Três coisas com o mesmo nome, e um método para escolher.

Hub, bibliotecas e ferramentas de agente

1 O Hub

Repositório de modelos, datasets e Spaces. É onde você ESCOLHE o modelo. Acessível pelo site e pela biblioteca `huggingface_hub`.

2 As bibliotecas

`transformers`, `sentence-transformers`, `datasets`, `peft`, `accelerate`. É onde o modelo RODA, inclusive dentro do seu processo Python.

3 Ferramentas de agente

`smolagents` e integrações MCP. O harness mais legível que existe hoje: núcleo pequeno, feito para ser lido.

Roteiro de escolha de modelo: faça ao vivo

1

Defina a tarefa e o orçamento de memória

text-generation? feature-extraction? zero-shot-classification? E quanta RAM ou VRAM você tem de verdade.

2

Verifique a licença antes de se apegar

Apache-2.0 e MIT são permissivas. Llama e Gemma têm licenças próprias com restrições. Isso importa se o projeto for comercial.

3

Leia o model card de verdade

Idiomas suportados, tamanho de contexto, se foi treinado para tool calling, e qual é o chat template.

4

Confira o formato e a atividade

safetensors para transformers, GGUF para Ollama e llama.cpp. Muitos downloads recentes indicam comunidade e correções.

Quanta memória o modelo vai ocupar

Precisão	Bytes por parâmetro	Modelo de 8B	Observação
FP16 / BF16	2	cerca de 16 GB	Qualidade de referência; exige GPU boa
INT8	1	cerca de 8 GB	Perda pequena, boa relação
Q4_K_M	cerca de 0,6	cerca de 5 GB	O padrão prático para rodar local
Q2	cerca de 0,4	cerca de 3 GB	Cabe, e a qualidade cai de forma visível

Filtrando o Hub programaticamente

O site é bom para explorar. O código é melhor para comparar e para deixar registrado no relatório o critério usado.

Conectando as duas metades do curso: o Hub é o catálogo, o Ollama é o runtime.

```
ollama pull hf.co/usuario/repo-gguf:Q4_K_M
```

```
from huggingface_hub import HfApi

api = HfApi()
for m in api.list_models(
    task="feature-extraction",
    library="sentence-transformers",
    sort="downloads", direction=-1, limit=10):
    print(f"{m.id:55} {m.downloads:>10}")

# rodando um GGUF do Hub no Ollama:
# $ ollama pull hf.co/<usuario>/<repo>:Q4_K_M
```

Lendo um model card com olhos de engenheiro

Seção	O que você procura	Sinal de alerta
Idiomas	Português está na lista de treino?	Só inglês, ou lista genérica sem detalhe
Contexto	Quantos tokens ele aguenta de verdade	Número grande sem menção a avaliação
Tool calling	Foi treinado para isso, e em que formato	Silêncio total sobre o assunto
Chat template	Existe e está no tokenizer_config	Ausente: você vai ter que montar à mão
Licença	Permissiva, ou com restrição de uso	Licença própria com cláusulas de uso

Ambiente isolado: virtualenv e Jupyter

Comparar embeddings, testar classificador, inspecionar tokenização: um notebook é mais rápido de iterar do que script mais terminal.

Por que um ambiente SEPARADO do resto do curso?
transformers e sentence-transformers trazem uma árvore de dependências pesada que não precisa estar nas máquinas que só falam com o Ollama por HTTP.

Notebook é para exploração. Quando o experimento funcionar, o código migra para um arquivo .py versionado.

```
$ python3 -m venv .venv-hf
$ source .venv-hf/bin/activate

$ pip install jupyter transformers \
    sentence-transformers huggingface_hub

$ jupyter notebook

# dentro do notebook:
from transformers import AutoTokenizer, pipeline

tok = AutoTokenizer.from_pretrained(
    "Qwen/Qwen3-0.6B")
clf = pipeline("sentiment-analysis")
```

Três decisões de escolha

1

Máquina com 8 GB de RAM, sem GPU

Qual o maior modelo viável, e em que quantização?

2

O produto será vendido a clientes

Qual é a primeira coisa a verificar no Hub?

3

A busca piorou depois de trocar o embedding

Qual linha do model card você não leu?

02

Abrindo a caixa

O que acontece antes do prompt chegar ao modelo.

Duas descobertas que a turma precisa fazer com as mãos

1. Português custa mais tokens que inglês. A mesma frase traduzida costuma gastar de vinte a quarenta por cento mais, o que significa contexto útil menor e resposta mais lenta.

2. Números são fatiados de formas estranhas. Um valor como 22800 pode virar dois ou três tokens, sem relação com os dígitos.

Isso não impede o modelo de calcular, e explica por que ele lida com número de um jeito diferente de como lida com palavra.

```
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6B")

ids = tok.encode("Sistemas multiagente com modelos locais")
print(len(ids))
print([tok.decode([i]) for i in ids])

print(len(tok.encode("950 vezes 24")))
print([tok.decode([i]) for i in tok.encode("22800")])
```

O que o custo de tokenização muda no seu projeto

Onde aparece	Efeito
Janela útil	Uma janela de 8k comporta menos conversa em português do que em inglês
Latência	Mais tokens de saída significam mais tempo, e em CPU isso é linear
Custo em API	Cobrança é por token; o mesmo texto custa mais caro em português
Chunking do RAG	800 caracteres em português dão mais tokens do que a intuição em inglês sugere
Limite de saída	max_tokens calibrado em inglês corta respostas em português no meio

O conceito mais subestimado do curso

A saída é o texto LITERAL que o modelo recebe, com os tokens especiais do formato. Ollama, llama.cpp e vLLM aplicam isso automaticamente, e é por isso que ninguém percebe que existe.

É por isso que: usar o template errado degrada o modelo sem dar erro nenhum; tool calling só funciona se o template souber renderizar ferramentas; e a flag --jinja do llama-server existe.

```
mensagens = [  
    {"role": "system", "content": "Voce e util."},  
    {"role": "user", "content": "Oi"},  
]  
  
print(tok.apply_chat_template(  
    mensagens, tokenize=False, add_generation_prompt=True))  
  
# <|im_start|>system  
# Voce e util.<|im_end|>  
# <|im_start|>user  
# Oi<|im_end|>  
# <|im_start|>assistant  
  
# o template tambem renderiza ferramentas:  
print(tok.apply_chat_template(  
    mensagens, tools=[schema_json], tokenize=False))
```

Sintomas de chat template errado

Sintoma	Causa provável	Correção
Modelo responde bem, mas ignora o system	Template sem suporte a system	Concatene as instruções na primeira mensagem
Tool calling não funciona nunca	Template sem renderização de ferramentas	Use --jinja, ou troque de modelo
Respostas com marcadores visíveis no texto	Template aplicado duas vezes	Não aplique manualmente se o servidor já aplica
Qualidade cai muito com o mesmo modelo	Template genérico em vez do próprio	Confira o tokenizer_config do repositório
O modelo continua o próprio texto	Faltou add_generation_prompt	Ative o parâmetro

Três perguntas

1

Por que o mesmo texto ocupa mais tokens em português?

E onde isso aparece no projeto?

2

O que o modelo realmente recebe?

Uma lista de dicionários, ou texto?

3

Tool calling funciona por causa de quê?

Do modelo, do servidor ou do template?

03

Pipelines prontos

As tarefas que já estão resolvidas.

A REGRA QUE MUDA O JULGAMENTO

Se a tarefa tem nome, já existe um modelo pequeno que a resolve melhor que um LLM.

Análise de sentimento tem nome. Extração de resposta tem nome. Classificação tem nome. Pesquisar sobre um assunto, cruzar com nossos documentos e escrever um relatório crítico não tem nome, e é aí que o agente ganha.

Cinco tarefas, cinco modelos, tempos de CPU

pipeline	Modelo típico	Tamanho	Latência
sentiment-analysis	nlptown/bert-base-multilingual	cerca de 670 MB	cerca de 30 ms
zero-shot-classification	MoritzLaurer/mDeBERTa-v3-base	cerca de 560 MB	cerca de 50 ms
question-answering	deepset/xlm-roberta-base-squad2	cerca de 1,1 GB	cerca de 40 ms
summarization	csebuetnlp/mT5_multilingual_XLSum	cerca de 2,3 GB	cerca de 300 ms
text-generation	Qwen/Qwen3-0.6B	cerca de 1,2 GB	cerca de 200 ms

Nem toda decisão precisa de um LLM.

Um classificador de trezentos megabytes que acerta 95% das vezes é melhor que um LLM de 4B que acerta 92% e demora sessenta vezes mais. Use o LLM onde é preciso generalizar; use modelo pequeno onde a tarefa é fechada. O roteamento inicial do supervisor é uma tarefa fechada.

Um classificador no lugar da primeira decisão

Responde em cerca de 50 milissegundos, contra cerca de 3 segundos de uma chamada ao LLM local. E é determinístico.

O campo `origem_decisao` permite medir depois quantas decisões foram baratas, e justificar a arquitetura com número, e não com opinião.

```
from transformers import pipeline

clf = pipeline("zero-shot-classification",
               model="MoritzLaurer/mDeBERTa-v3-base-mnli-xnli")

r = clf("Quanto foi o faturamento do trimestre?",
        candidate_labels=["pesquisa", "calculo",
                          "escrita", "conversa"])
print(r["labels"][0], round(r["scores"][0], 3))

def no_supervisor(estado):
    if estado.get("rodadas", 0) == 0:
        rot, nota = rotear_barato(estado["pergunta"])
        if rot:
            return {"proximo": rot,
                    "origem_decisao": "classificador"}
    ... # so entao gasta uma chamada de LLM
```

Um parâmetro separa o script útil do inutilizável

Processar mil avaliações uma a uma leva minutos. Em lotes de dezesseis, leva segundos.

`truncation=True` evita que uma avaliação longa derrube o script inteiro. Presente em todo processamento de dado real.

```
import pandas as pd
from transformers import pipeline

analizador = pipeline(
    "sentiment-analysis",
    model="nlpTown/bert-base-multilingual-uncased-sentiment")

df = pd.read_csv("dados/avaliacoes.csv")

resultados = analizador(df["texto"].tolist(),
                        batch_size=16,
                        truncation=True)

# o rótulo é string ("4 stars"), não número:
# cada modelo rotula à sua maneira
df["estrelas"] = [int(r["label"][0]) for r in resultados]
df["confiança"] = [round(r["score"], 3) for r in resultados]

print(df.groupby("produto")["estrelas"]
      .agg(["mean", "count"]).sort_values("mean"))
```

O corte por score é a lição de arquitetura

Classificar produtos em seis categorias sem ter treinado nada para isso. O mesmo pipeline do roteamento, num problema de negócio.

Um classificador que sempre devolve alguma categoria é perigoso. Um que separa o que não sabe e manda para revisão humana é utilizável.

```
categorias = ["eletronicos", "vestuario",
             "alimentos", "casa e decoracao",
             "esporte", "livros"]

clf = pipeline("zero-shot-classification",
              model="MoritzLaurer/mDeBERTa-v3-base-mnli-xnli")

def categorizar(descricao: str) -> tuple[str, float]:
    r = clf(descricao, candidate_labels=categorias)
    return r["labels"][0], round(r["scores"][0], 3)

df["categoria"], df["score"] = zip(
    *df["descricao"].map(categorizar))

# score baixo = a categoria certa nao esta na lista
duvidosos = df[df["score"] < 0.5]
print(f"{len(duvidosos)} itens precisam de revisao")
```

RAG com LLM contra question answering extrativo

L RAG com LLM

- Resposta GERADA pelo modelo
- Pode alucinar, mesmo com contexto
- Responde o que não está lá, inventando
- Reformula, resume, junta fontes
- 2 a 5 segundos

E QA extrativo

- Resposta RECORTADA do texto
- Não alucina: só copia
- Devolve score baixo quando não sabe
- Não reformula nem resume
- cerca de 40 milissegundos

Rápido no caso comum, capaz no caso difícil

Junte os dois: recuperação semântica para reduzir o universo, extração para responder, geração só quando necessário.

Meça quantas das dez perguntas foram resolvidas sem chamar o LLM nenhuma vez. Esse número é o valor do exercício.

```
qa = pipeline("question-answering",
              model="deepset/xlm-roberta-base-squad2")

CORTE = 0.35

def responder(pergunta: str) -> str:
    trechos = banco.similarity_search(pergunta, k=3)
    contexto = "\n\n".join(d.page_content for d in trechos)

    r = qa(question=pergunta, context=contexto)

    if r["score"] >= CORTE:
        return f"{r['answer']} [citacao literal]"

    # so agora o LLM entra
    return llm.invoke([
        SystemMessage(f"Responda usando so:\n{contexto}"),
        HumanMessage(pergunta)]).content
```

Laboratório da manhã: substituir uma decisão

1

Montar o ambiente e rodar dois pipelines

sentiment e zero-shot, cronometrados, com o modelo carregado uma vez só.

2

Trocar a primeira decisão do supervisor

Classificador zero-shot com corte por confiança, e queda para o LLM abaixo do corte.

3

Medir a concordância em vinte perguntas

Classificador contra LLM. Anote a taxa e o tempo médio de cada um.

4

Registrar a origem de cada decisão

O campo `origem_decisao` no estado, e a contagem ao fim da execução.

5

Extensão: o FAQ híbrido

QA extrativo antes do LLM, com duas perguntas fora da base para testar as recusas.

04

Embeddings, rerank e avaliação

Escolher com critério o que até agora foi escolhido no chute.

Escolhendo um modelo de embedding

Critério	Por que importa
Multilíngue	A base do aluno está em português; modelo só de inglês perde muito
Dimensão do vetor	384 é mais rápido e menor que 1024, com pouca perda em base pequena
Tamanho máximo de entrada	Se o chunk tem 800 caracteres, 512 tokens já basta
Prefixos de instrução	Algumas famílias pedem prefixo em consulta e em passagem. Ignorar isso derruba a qualidade sem dar erro
Licença	Mesmo critério dos LLMs; verifique antes de embutir no produto

Rerank: a melhoria de RAG com melhor relação custo-benefício

Busca vetorial é rápida e imprecisa. Um cross-encoder é lento e preciso. Combine: busque vinte, reordene, entregue quatro.

O ganho de qualidade nos trechos costuma ser grande, e o custo é baixo porque o cross-encoder só vê vinte pares, e não a base inteira.

Meça antes e depois nas mesmas cinco perguntas: é o exercício.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder(
    "cross-encoder/mmarco-mMiniLMv2-L12-H384-v1")

def buscar_com_rerank(consulta, k=4):
    brutos = banco.similarity_search(consulta, k=20)
    pares = [(consulta, d.page_content) for d in brutos]
    notas = reranker.predict(pares)
    ordenados = sorted(zip(notas, brutos),
                       key=lambda x: -x[0])
    return [d for _, d in ordenados[:k]]
```

Reformulação de query

Rerank melhora a ORDENAÇÃO dos candidatos.

Reformulação melhora o que entra na busca ANTES de ela acontecer. A pergunta do usuário raramente é a melhor consulta para um vector store: ela é conversacional, tem pronomes e referências implícitas.

Uma pergunta como e o de 2023, num histórico sobre faturamento de 2024, vira faturamento da Cooperativa Vale Verde em 2023.

Reformulação, busca e rerank atacam pontos diferentes do mesmo pipeline e se combinam sem conflito.

```
PROMPT = """Reescreva a pergunta como consulta  
autocontida, sem pronomes nem referencias.  
Devolva so a consulta.
```

```
Historico: {historico}  
Pergunta: {pergunta}"""
```

```
def reescrever(llm, pergunta, historico):  
    return llm.invoke(PROMPT.format(  
        historico=historico,  
        pergunta=pergunta)).content.strip()
```

DUAS FERRAMENTAS, DOIS PROBLEMAS

Observabilidade é o que o pipeline FEZ.
Avaliação é se o que ele fez estava CERTO.

Até aqui medimos que funcionou numa pergunta. Um sistema em produção precisa de medição contínua dos dois lados, e são ferramentas diferentes, não a mesma coisa com nomes diferentes. O traço do dia 4 responde a primeira; o conjunto de avaliação responde a segunda.

Três métricas, três tipos de falha

Métrica	Pergunta que responde	Pega qual falha
faithfulness	A resposta é sustentada pelo contexto recuperado?	Alucinação mesmo com RAG: é o prompt do gerador
answer_relevancy	A resposta de fato responde à pergunta?	Resposta correta mas fora do assunto ou incompleta
context_precision	Os trechos recuperados eram relevantes?	Falha de busca: chunking, embedding ou k mal ajustados

Dez perguntas, e a tabela que decide

```
# avaliacao/casos.jsonl - uma linha por caso
{"id": "q1", "pergunta": "Qual foi o faturamento de 2024?",
 "esperado": "R$ 4,2 milhoes", "fonte": "2024_relatorio.md"}
{"id": "q2", "pergunta": "Some 2023 e 2024 e diga a media.",
 "esperado": "R$ 3,85 milhoes", "fonte": "2023_relatorio.md,2024_relatorio.md"}
{"id": "q3", "pergunta": "Quem e o presidente da cooperativa em 2030?",
 "esperado": "RECUSAR", "fonte": ""}

# rodar e comparar
para cada caso:
    resposta, fontes, tokens, segundos = nexus(caso["pergunta"])
    acertou = confere(resposta, caso["esperado"])
    citou_ok = set(fontes) >= set(caso["fonte"].split(","))
    registrar(caso["id"], acertou, citou_ok, tokens, segundos)
```

O caso q3 é o mais importante do conjunto: a resposta certa é recusar. Um sistema que nunca recusa não é confiável, é apenas confiante.

LLM como juiz, e os cuidados

1

Serve quando a resposta é texto livre

Comparar string não funciona para um relatório de trezentas palavras. Um juiz com rubrica funciona.

2

A rubrica precisa ser verificável

Cada critério respondido com sim ou não, olhando o texto. Nota de zero a dez sem rubrica é ruído.

3

Use um modelo diferente do que gerou

Juiz e gerador iguais tendem a concordar consigo mesmos. É o mesmo argumento do Crítico independente do dia 4.

4

Calibre contra julgamento humano

Rotule vinte casos à mão e compare com o juiz. Se a concordância for baixa, conserte a rubrica antes de confiar.

Laboratório da tarde: medir para decidir

1

Trocar o modelo de embedding e reindexar

Escolha por critério, leia o card, e verifique se ele exige prefixo.

2

Ligar o rerank em dois estágios

Buscar vinte, reordenar, entregar quatro. Meça o tempo antes e depois.

3

Escrever dez casos de avaliação

Com pelo menos dois cuja resposta correta é recusar.

4

Rodar a bateria antes e depois do rerank

Quatro colunas: acertou, citou a fonte certa, tokens, segundos.

5

Concluir com número, não com impressão

Uma frase: o rerank mudou o acerto de X para Y e custou Z segundos a mais.

05

smolagents e deploy

Ler um harness inteiro, e colocar algo no ar.

CodeAgent: o modelo escreve código, não JSON

A diferença conceitual: o agente não emite tool call em JSON. Ele escreve código Python que chama as ferramentas, e o harness executa esse código num sandbox.

Vantagem: composição natural, com laços, condicionais e variáveis intermediárias, e menos idas e voltas.

Desvantagem: executar código gerado por LLM exige sandbox de verdade. Por isso o smolagents oferece execução em Docker ou em serviço isolado.

```
from smolagents import CodeAgent, TransformersModel, tool

@tool
def calcular(expressao: str) -> str:
    """Avalia uma expressao aritmetica.

    Args:
        expressao: a expressao, ex '2+2'
    """
    ...

modelo = TransformersModel("Qwen/Qwen3-1.7B")
agente = CodeAgent(tools=[calcular], model=modelo)
agente.run("Some os numeros de 1 a 100 e divida por 7")

# o modelo gera:
# soma = sum(range(1, 101))
# print(calcular(f"{soma} / 7"))
```

Vinte minutos lendo um harness inteiro

1

Localize o loop

Ele existe, é curto, e é reconhecível. Compare com as suas sessenta linhas do dia 1.

2

Localize as condições de parada

Quantas são? Onde estão? São as mesmas quatro que você implementou?

3

Localize a montagem de contexto

Onde o prompt final é montado, e o que entra nele além das mensagens.

4

Localize o ponto de execução da ferramenta

É ali que um PreToolUse entraria. Existe algum gancho pronto?

5

Localize o sandbox

Como o código gerado é isolado, e o que acontece quando não há isolamento configurado.

A mesma metodologia, num domínio que não foi ensinado

Remoção de fundo é tarefa de visão, resolvida por modelos de segmentação. O ponto pedagógico não é a imagem.

Escolher no Hub por tarefa, ler o card, instalar, chamar, avaliar. O aluno acabou de aplicar a metodologia da manhã a algo que não foi ensinado, que é o que se espera dele depois do curso.

```
# pip install rembg pillow onnxruntime

from rembg import remove
from PIL import Image

def remover_fundo(entrada: str, saida: str) -> str:
    img = Image.open(entrada)
    resultado = remove(img) # RGBA, fundo transparente
    resultado.save(saida)
    return saida

# a interface, para quem nao tem Python:
import gradio as gr

demo = gr.Interface(
    fn=lambda img: remove(img),
    inputs=gr.Image(type="pil"),
    outputs=gr.Image(type="pil"),
    title="Removedor de fundo")

demo.launch()
```

Três arquivos e um git push

O Spaces hospeda de graça no tier de CPU. O cabeçalho YAML do README é o que ele lê como configuração.

Se o Space ficar preso em Building, confira o `sdk_version`: uma versão que não existe trava o build sem mensagem clara.

```
meu-space/  
  app.py          # a interface Gradio  
  requirements.txt # gradio, rembg, pillow  
  README.md       # com o cabeçalho abaixo  
  
# --- README.md ---  
---  
title: Removedor de Fundo  
emoji: camera  
colorFrom: indigo  
colorTo: purple  
sdk: gradio  
sdk_version: 5.9.1  
app_file: app.py  
---  
  
# --- terminal ---  
huggingface-cli login  
git init  
git remote add origin \  
  https://huggingface.co/spaces/USUARIO/removedor  
git add . && git commit -m "primeira versao"  
git push origin main
```


Dois avisos sobre o Spaces

Aviso	Por quê	O que fazer
O tier gratuito é CPU com 16 GB	Serve para modelos pequenos; um LLM de 4B não cabe confortavelmente	Não publique o NEXUS lá; só a demonstração pequena
Um Space é um repositório Git	Uma chave comitada é uma chave vazada, e o histórico guarda	Use os secrets do repositório. Nunca no código

Quatro destinos, quatro perfis

Destino	Bom para	Cuidado principal
Máquina do usuário	Ferramenta interna, dado sensível	Cada máquina precisa do modelo e da manutenção
Servidor próprio com GPU	Equipe inteira, vLLM servindo vários agentes	Fila, concorrência e memória compartilhada
Container em nuvem privada	Empresa que já tem plataforma	Custo de GPU ociosa; considere autoescala
Serviço gerenciado de agentes	Quem não quer operar infraestrutura	O dado sai da sua rede; verifique antes

FECHO DO BLOCO

Um projeto que só roda no seu terminal não existe para mais ninguém.

Não importa se o destino é um Space gratuito, um container interno ou o notebook de um colega: em algum momento outra pessoa precisa conseguir usar o que você construiu, sem você por perto. Esse é o teste final de qualquer sistema.

06

O NEXUS final

Um modelo por papel, o método de depuração e o fechamento.

Um modelo por papel, com justificativa

Papel	Modelo	Temp.	Por quê
supervisor	qwen3:1.7b	0	Roda a cada transição; decisão curta e estruturada. É onde modelo pequeno economiza mais
pesquisador	qwen3:4b	0	Precisa seguir instrução e ler trechos longos
analista	qwen3:1.7b	0	Só orchestra a ferramenta calcular; a conta não é dele
redator	qwen3:8b	0.3	Roda uma vez; vale o modelo mais capaz. Temperatura maior evita texto repetitivo
critico	qwen3:4b	0	Deve ser rigoroso e determinístico

A ficha de escolha de modelo

1

Qual é a tarefa deste papel

Em uma frase, e sem adjetivos. Decidir entre cinco opções é diferente de escrever quatrocentas palavras.

2

Quantas vezes ele roda por execução

O que roda a cada transição merece o modelo mais barato que resolve.

3

O que quebra se o modelo for pequeno demais

Structured output inválido? Texto pobre? Escolha errada de ferramenta? Cada falha aponta um requisito.

4

Qual foi a medição

Vinte execuções, taxa de acerto, tempo médio. É a linha que transforma escolha em decisão.

O que fazer quando não funciona, nesta ordem

1

O modelo está recebendo o que você acha?

Imprima o prompt final, com template aplicado. Metade dos problemas morre aqui.

2

A ferramenta funciona sozinha, no REPL?

Chame invoke direto na ferramenta. Se falha aqui, não é problema de agente.

3

O modelo consegue chamar a ferramenta?

bind_tools e olhe tool_calls. Vazio? É o modelo, o servidor ou o schema.

4

O roteamento e o estado estão certos?

Teste o roteador com estados falsos, sem LLM. Campo vazio no estado costuma ser reducer faltando.

5

Quantos passos e quantos tokens?

Explodiu: é loop. Muito curto: é parada prematura. Só DEPOIS disso tudo, mexa no prompt.

De sessenta linhas a um sistema

1

loop escrito à mão, no dia 1

5

papéis no sistema final

3

modelos diferentes por função

0

chamadas a API paga

Volte ao dia 1 e releia as sessenta linhas do primeiro agente. Tudo o que veio depois, LangChain, LangGraph, supervisor, Send, checkpointer, Hugging Face, são camadas sobre aquele loop.

As cinco camadas, ao fim da semana

Camada	Onde ela está no NEXUS final
Instructions	Cinco prompts de papel, descriptions de ferramenta, ordem de preferência entre elas
Constraints	Hooks de PreToolUse, sandbox de caminho, limite de passos e de rodadas, aprovação humana
Feedback	O Crítico com rubrica, o retry de validação, os testes sem LLM, o conjunto de avaliação
Memory	Checkpointter com thread_id por dono, store semântico, poda de contexto
Orchestration	Supervisor com especialistas, paralelismo por Send, um modelo por papel

O que este curso NÃO cobriu

Não cobrimos	Onde continuar
Avaliação em escala	Benchmarks, conjuntos de regressão grandes, avaliação contínua em produção
RAG avançado	GraphRAG, busca híbrida com BM25, HyDE
Fine-tuning e LoRA	peft, trl e as bibliotecas de treino
Segurança em profundidade	Injeção de prompt, sandbox real, princípio do menor privilégio
Produção	Filas, retries, custo, SLO, autoescala de GPU
Escrever um servidor MCP	Nós consumimos MCP; escrever o seu é o passo seguinte
Agentes de computador	Navegador, interface gráfica, harnesses focados em repositório

Como continuar estudando com método

1

Leia o código de um harness pequeno, inteiro

smolagents é o melhor candidato. Ler um sistema completo ensina mais que dez tutoriais.

2

Reimplemente uma peça do zero

Nada ensina como reescrever o ToolNode e comparar com o original.

3

Mantenha o caderno de falhas

Ele é o seu conjunto de testes de regressão informal, e o insumo do próximo projeto.

4

Acompanhe changelogs, não influenciadores

LangGraph, vLLM e transformers têm notas de versão boas e honestas.

5

Meça antes de opinar

Dizer que um modelo é melhor para agente sem uma tabela é ruído.

OS SETE PRINCÍPIOS

O que levar como princípio, não como código

1. O modelo é uma peça; o sistema é o harness. 2. Contexto é recurso escasso: gaste-o com intenção. 3. Ferramenta boa erra com mensagem útil. 4. Todo ciclo precisa de um limite explícito. 5. Nem toda decisão precisa de LLM. 6. Comece com um agente; divida com evidência. 7. Meça antes de opinar.

Dez dias, um domínio, três agentes

1

Escolha um domínio real, não pesquisa genérica

Jurídico, financeiro, suporte, compras, acadêmico, operações. O NEXUS já é pesquisa genérica.

2

Mínimo: 3 agentes, 5 ferramentas, 2 modelos, 1 componente não-LLM

Tudo local. Nenhuma chamada a API paga.

3

Dez perguntas de teste com resposta anotada à mão

E a tabela de resultados, com pelo menos duas perguntas cuja resposta certa é recusar.

4

Um documento de análise com três falhas em profundidade

O que aconteceu, a hipótese, o que você tentou, o que funcionou. É o que separa o trabalho bom do mediano.

5

Na apresentação, mostre uma pergunta que o sistema erra

Mostrar uma falha vale mais do que escondê-la. Sistemas de agente falham; a competência está em saber por quê.